

Managing Linux Files and Directories

In this chapter, you will learn about

- Linux file systems and the Filesystem Hierarchy Standard (FHS)
- Finding files in the Linux file system
- Managing directories from the command line
- Managing files from the command line
- Working with link files
- Finding content within files

It's time to dig in and start working with Linux files and directories! If you're coming to Linux from a Windows background, this chapter may be challenging for you. The Linux file system is similar to the DOS or Windows file systems in many ways, but it is radically different in many others. My experience with new Linux students is that there are just enough similarities to give them a false sense of security when they initially start working with the file system. Then, as we dig deeper and start looking at more advanced tasks, they start to get frustrated.

Hopefully, I can make things as easy to understand as possible as you work through this chapter. If I can emphasize one thing that will make your life easier, it would be to remember that Linux file and directory names are case sensitive! New Linux users tend to really struggle with this aspect of the Linux file system. When you see a command or filename in an example or exercise in this book, remember that you must use the correct case; otherwise, you won't get the same results!



EXAM TIP Be prepared to be tested very heavily on this chapter in your Linux exam. If you haven't set up a lab system to practice on yet, I strongly recommend that you do so before proceeding. Then practice completing the examples and exercises presented in this chapter over and over until you feel thoroughly comfortable with the tasks and utilities involved. When you take your exam, you may be presented with a hypothetical scenario and asked to supply the correct command to complete the required task. You must be familiar enough with the contents of this chapter to respond instantly. The only way to do this is to practice, practice, practice!

Linux File Systems and the Filesystem Hierarchy Standard (FHS)

In this part of the chapter, we're going to discuss the Linux file system in general. Specifically, we're going to review the following:

- The role of the Linux file system
- The hierarchical structure of the Linux file system
- Linux file types

Let's begin by discussing the role of the file system.

The Role of the Linux File System

So what exactly is the general role of the file system on a Linux system (or any other operating system, for that matter)? We're talking about a system that stores information on a storage device in such a manner that

- Data can be saved in a logical manner.
- Data is organized and can be easily located.
- Data can be retrieved for use at a later point in time.
- Data integrity is preserved.

In other words, if you save a file to a storage device you should be able to find it later on and retrieve it, all the while assured that its contents will be exactly the same as when it was saved. It doesn't matter whether you're saving data to a floppy diskette, a hard drive, or a USB drive; the goals are the same. To accomplish this, you must use some type of file system to manage stored data.



TIP The term *file system* actually has multiple meanings in the Linux world. You may hear the terms "ext2," "ext3," "ext4," and "Reiser" referred to as *file systems*. That usage is correct, they are file systems. However, they are more properly identified as *disk file systems*. They are used to define how data is stored on your Linux system's hard disk drive. A disk file system is a specific implementation of the general file system. Specifics about each type of file system will be discussed in Chapter 5.

Let's review how the Linux file system organizes the data to make it easily locatable and retrievable using the Filesystem Hierarchy Standard (FHS).

The Hierarchical Structure of the Linux File System

The Linux file system uses a hierarchical structure to organize and store data. This is shown in Figure 4-1.

The topmost directory in the structure is the `/` directory, also called the *root* directory, which by the way has nothing to do with your root user account. It simply indicates that this is the topmost directory (the "root") of your hierarchical file system tree.

Beneath the root directory are many subdirectories. Specifications for how these directories are to be named are contained in the FHS. The FHS provides Linux software developers and system administrators with a standard directory structure for the file system, ensuring consistency between systems and distributions. Can you imagine the mess if every distribution used its own proprietary files system structure? What a disaster! You wouldn't be able to find anything.

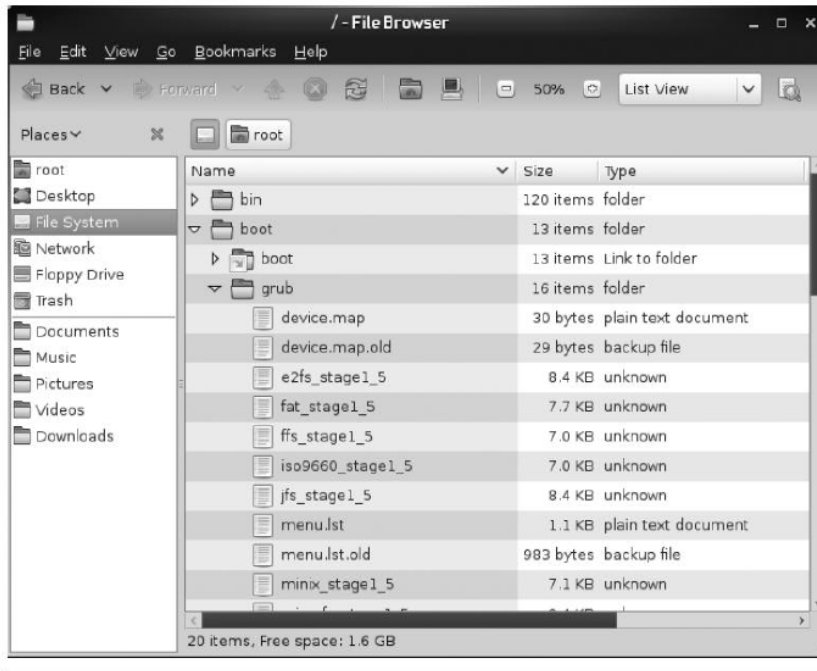


Figure 4-1 The hierarchical structure of the Linux file system

Fortunately, we have the FHS to keep everyone in line. The FHS defines the directories that should appear under the root directory (/) as well as the directories that should appear under the /usr and /var directories. These include the following:

- /bin This directory contains executable files necessary to manage and run the Linux system, including shells (such as bash) and file system management utilities such as cp and rm.
- /boot This directory contains your bootloader files, which are required to boot your system.
- /dev This directory contains special files that are used to represent the various hardware devices installed in the system. For example, the first hard disk drive in your system is called sda, the second is called sdb, and so on. The partitions on each drive are identified by an additional number added to the end of the disk filename. For example, the first partition on the first hard drive in the system is sda1, the second partition is sda2, and so on. The files that represent these devices are stored in /dev, as shown in Figure 4-2. Notice in the figure that two kinds of device files are stored in /dev:
 - Character-oriented device files These files are used for devices that send or receive data sequentially one character at a time, such as a printer, mouse, or tape drive. These devices usually don't support random access to the data they manage.
 - Block-oriented device files These files are used for devices that manage data in blocks, such as floppy disks and hard drives. Block devices usually support random access to the data they manage.

Many types of hardware devices are represented by device files in /dev. A sampling is shown in Table 4-1. The key thing to understand is the fact that the physical hardware in a Linux system is addressed by applications and services running on the system through the files in /dev. Need to save a file to disk? It goes through the appropriate file in /dev. Need to send a print job to a printer? It goes through a file in /dev. Need to open a file from a DVD? It comes in through a file in /dev.



TIP In older versions of the Linux kernel, SCSI, USB, and SATA hard drives were denoted by sdx while older PATA (IDE) hard drives were denoted by hdx. This is no longer the case. Now all hard disks are denoted by sdx.

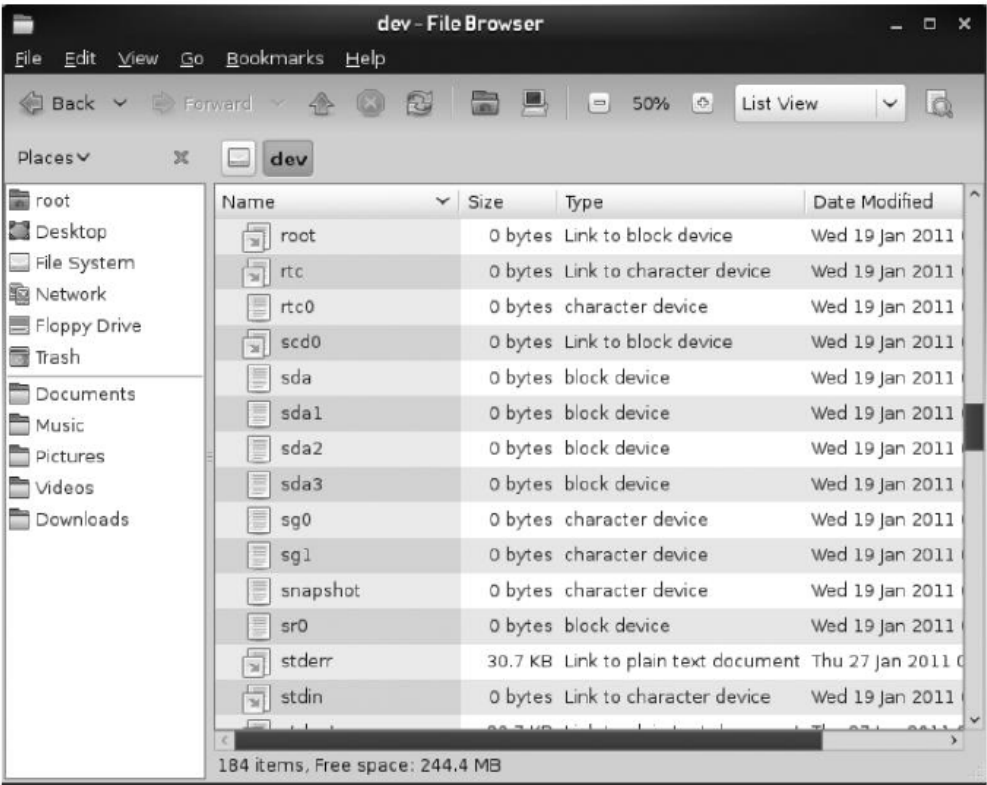


Figure 4-2 Addressing hardware devices with files in /dev



TIP Device files in /dev can also be used to access system resources that have no connection to any physical hardware device, such as random number generators (/dev/random).

Table 4-1
Hardware
Represented
by Files in /dev

Device	Device File in /dev
Floppy drive	/dev/fd0
Optical drive (CD or DVD)	/dev/scd0
Serial port	/dev/ttyS0
Parallel port	/dev/lp0

- **/etc** This directory contains text-based configuration files used by the system, as well as services running on the system. You can edit these files with a text editor to customize how Linux behaves. Some of the more important files in this directory include those shown in Table 4-2.
- **/home** This directory contains subdirectories that serve as home directories for each user account on your Linux system.
- **/lib** This directory contains code libraries used by programs in **/bin** and **/sbin**. Your kernel modules are also stored in the **modules** subdirectory of **/lib**.
- **/media** This directory is used by some Linux distributions (such as openSLISE and Fedora) to mount external devices, including optical drives, LISB drives, and floppy diskettes (if you're still using them). This is done using a series of subdirectories, shown in Figure 4-3.
- **/mnt** This directory is used by some Linux distributions to mount external devices, including CD drives, DVD drives, LISB drives, and floppies. As with the **/media** directory on other distributions, a series of subdirectories are used to do this.

File	Function
/etc/aliases	Contains a table used to redirect mail to local recipients.
/etc/exports	Configures file systems to be exported to remote NFS clients.
/etc/fstab	Lists the partitions and file systems that will be automatically mounted when the system boots.
/etc/ftpusers	Controls user access to the FTP service running on the system.
/etc/group	Contains local group definitions.
/etc/grub.conf	Contains configuration parameters for the GRUB bootloader.
/etc/hosts	Contains a list of hostname-to-IP address mappings the system can use to resolve hostnames.
/etc/inittab	Contains configuration parameters for the init process.
/etc/init.d/	A subdirectory that contains startup scripts for services installed on the system. On a Fedora or Red Hat system, these are located in /etc/rc.d/init.d .
/etc/modules.conf	Contains configuration parameters for your kernel modules.
/etc/nsswitch.conf	Configures which services are to be used to resolve hostnames and to store users, groups, and passwords.
/etc/passwd	Contains your system user accounts.
/etc/resolv.conf	Specifies the DNS server and domain suffix used by the system.
/etc/services	Maps port numbers to named services on the system.
/etc/shadow	Contains encrypted passwords for your user accounts.
/etc/X11/	Contains X Window configuration files.

Table 4-2 Some of the Configuration Files in **/etc**

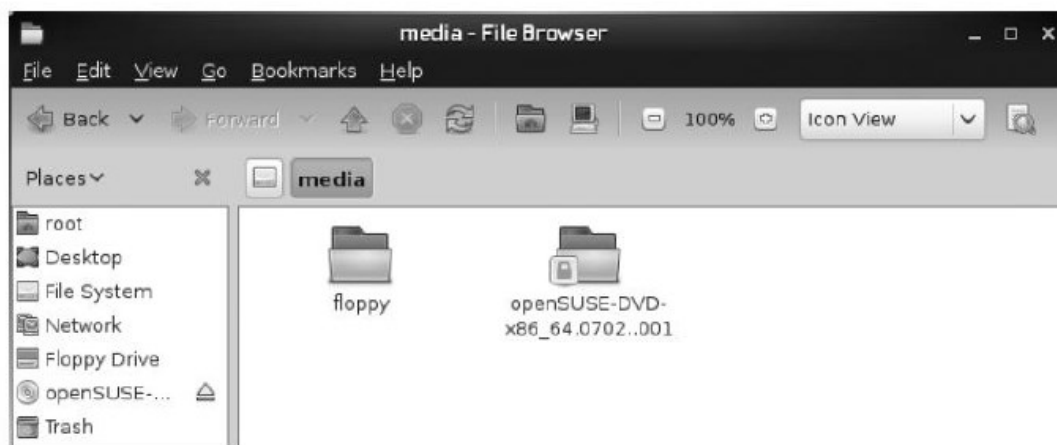


Figure 4-3 Subdirectories of /media

- /opt This directory contains files for some programs you install on the system.
- /proc This directory is a little different from the other directories in this list, /proc doesn't actually exist in the file system. Instead, it's a pseudo-file system that is dynamically created whenever it is accessed. It's used to access process and other system information from the Linux kernel.

Within /proc are a number of different subdirectories, as shown in Figure 4-4. Notice that each of these subdirectories is identified with a number, not a name. These numbers correspond to the process ID (PID) number of the associated process running on the system. In Figure 4-5, the top program is being run to display the running processes on the system.

Figure 4-4 Subdirectories of /proc

fs2:/proc # ls									
1	1807	2378	35	776	9186	9267	ioports	schedstat	
10	1000	2309	36	777	9194	9200	irq	scsi	
10040	1062	24	38	778	9198	9271	kallsyms	self	
10150	1069	2581	39	7871	9204	9278	kcrc	slabinfo	
10159	1889	2611	4	79	9205	9289	kdb	softirqs	
10160	19	27	419	8	9208	9323	key-users	splash	
10178	1934	2754	4250	80	9218	acpi	kmsg	swaps	
10170	1037	2755	4487	8568	9223	asound	kpagedcount	etap	
10190	1954	2756	4548	8998	9225	buddyinfo	kpagelflags	sys	
11	2	2757	5	8999	9228	bus	latency_stats	sysrq-trigger	
12	20	2758	584	9	9236	cmoline	loadavg	sysvipc	
1243	2016	2759	6	9025	9238	config.gz	locks	timer_list	
1251	2032	28	600	9045	9242	cpuinfo	mdstat	timer_stats	
13	2067	29	68	9050	9243	crypto	meminfo	tty	
1331	2074	297	6876	9066	9246	devices	misc	uptime	
1333	21	298	69	9086	9248	diskstats	modules	version	
14	2107	299	7	9130	9251	cma	mounts	vmallocinfo	
15	22	3	70	9145	9253	driver	mpt	vmstat	
1581	2240	30	7135	9149	9254	execdomains	mtrr	zoneinfo	
16	2247	305	7136	9153	9257	fb	net		
17	2252	31	72	9160	9258	filesystems	pagetypeinfo		
1780	2253	32	726	9162	9260	fs	partitions		
18	2283	33	73	9168	9262	interrupts	reserve_info		
1802	23	34	74	9172	9265	ionem	sched_debug		
fs2:/proc #									

Figure 4-5
Using top to display
running processes

```
top - 15:06:45 up 11:08, 4 users, load average: 0.03, 1.59, 1.64
Tasks: 149 total, 1 running, 148 sleeping, 0 stopped, 0 zombie
Cpu(s): 0.2%us, 0.3%sy, 0.0%ni, 99.3%id, 0.2%wa, 0.0%hi, 0.0%si, 0.0%st
Mem: 503700k total, 400196k used, 37304k free, 22190k buffers
Swap: 1532924k total, 8856k used, 1524068k free, 223140k cached
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
9254	root	20	0	379m	30m	16m	S	1	6.1	0:03.24	gnome-do
10158	root	20	0	286m	17m	11m	S	0	3.5	0:01.20	gnome-terminal
10178	root	20	0	17088	1284	900	R	0	0.3	0:00.19	top
1	root	20	0	12404	636	600	S	0	0.1	0:03.33	init
2	root	20	0	0	0	0	S	0	0.0	0:00.01	kthreadd
3	root	RT	0	0	0	0	S	0	0.0	0:00.17	migration/0
4	root	20	0	0	0	0	S	0	0.0	0:00.48	ksoftirqd/0
5	root	RT	0	0	0	0	S	0	0.0	0:00.00	watchdog/0
6	root	RT	0	0	0	0	S	0	0.0	0:00.13	migration/1
7	root	20	0	0	0	0	S	0	0.0	0:00.85	ksoftirqd/1
8	root	RT	0	0	0	0	S	0	0.0	0:00.00	watchdog/1
9	root	20	0	0	0	0	S	0	0.0	0:02.97	events/0
10	root	20	0	0	0	0	S	0	0.0	0:03.86	events/1
11	root	20	0	0	0	0	S	0	0.0	0:00.00	netns
12	root	20	0	0	0	0	S	0	0.0	0:00.00	async/nqr
13	root	20	0	0	0	0	S	0	0.0	0:00.00	pm
14	root	20	0	0	0	0	S	0	0.0	0:00.08	sync_supers
15	root	20	0	0	0	0	S	0	0.0	0:00.08	bdi-default
16	root	20	0	0	0	0	S	0	0.0	0:00.00	kintegrityd/0

The PID column, on the far-left side of the display, lists the PID number of each process. The actual name of the command used to create the process is displayed in the far-right column of the display. For example, the process associated with the top program itself has a PID of 10178. Therefore, if you were to look in the /proc/10178 directory, you could view information about the top process running on the system, as shown in Figure 4-6.

- /root This directory is the root user's home directory. Notice that it is located separately from the home directories for other users in /home.
- /sbin This directory contains important system management and administration files, such as fdisk, fsck, ifconfig, init, mkfs, shutdown, and halt.
- /srv This directory contains subdirectories where services running on the system (such as httpd and ftpd) save their files.
- /sys This directory contains information about the hardware in your system.
- /tmp This directory contains temporary files created by you or by the system.



TIP The system will periodically delete old files out of the /tmp directory. Don't save anything in /tmp that you want to keep!

Figure 4-6
Viewing information
about the top
process in /proc

```
fs2:/proc/10178 # ls
attr          cwd           latency      mounts        pagemap      smaps         task
auxv          environ      limits       mountstate    personality   stack         wchan
clear_refs    exe          loginuid     net           root         stat
cmdline       fd           maps         num_maps     sched        statm
comm          fdinfo       mem          oom_adj       schedstat    status
coredump_filter io            mountinfo    oom_score     sessionid    syscall
fs2:/proc/10178 #
```

Subdirectory	Contents
bin	Most of your executable programs
lib	Library files
lib64	64-bit library files
local	Locally installed software that you created yourself (used to prevent it from being overwritten during a system update)
sbin	System administration programs
share	Documentation and man page files

Table 4-3 Subdirectories of /usr

- /usr This directory contains application files. In fact, most of the application files used on your system are saved in a subdirectory of /usr. These subdirectories include those shown in Table 4-3.
- /var This directory contains variable data, including your system log files. Some of the typical subdirectories contained in /var are shown in Table 4-4.

With this structure in mind, let's next discuss the types of files used by the Linux file system.

Types of Files Used by Linux

When working with Linux, you need to be aware of the fact that there are a number of different file types used by the file system. This is another area where the Linux file system differs significantly from the Windows file system. With a Windows file system you basically have two entry types in the file system:

- Directories
- Files

Granted, you can have normal files, hidden files, shortcut files, word processing files, executable files, and so on. However, these are all simple variations of the basic Windows file.

On Linux, however, there are a variety of different file types used by the file system. These include the file types listed in Table 4-5.

Subdirectory	Contents
lib	Library files created by various services and applications running on the system
log	Log files from your system and from services running on the system
spool	Print queues

Table 4-4 Subdirectories of /var

File Type	Description
Regular files	These files are similar to those used by the file systems of other operating systems—for example, executable files, word processing files, images, text files, etc.
Links	These files are pointers that point to other files in the file system.
FIFOs	FIFO stands for <i>First In First Out</i> . These are special files used to move data from one running process on the system to another. A FIFO file is basically a queue where the first chunk of data added to the queue is the first chunk of data removed from the queue. Data can only move in one direction through a FIFO.
Sockets	Sockets are similar to FIFOs in that they are used to transfer information between processes. Unlike FIFOs, however, sockets can move data bidirectionally.

Table 4-5 Linux File Types

With this introduction to the Linux file system in mind, you're ready to start learning how to complete common file system tasks. Just as with any other operating system, you need to be able to create, copy, and move files and directories in the Linux file system. In this part of this chapter, we're going to spend some time reviewing how this is done.

If you have any experience working with the command line in other operating systems, such as DOS or Windows, you may be thinking that these tasks are trivial in nature. Believe it or not, these tasks can be difficult for many of my students when they are first learning how to use Linux. The key mistake I see is students trying to use Windows (and DOS) commands when working with the file system at the Linux shell prompt.

I even catch myself doing this on occasion. I was presenting a demo for students recently and needed to copy some files on a Linux system. Without thinking, I entered `copy` instead of `cp` at the shell prompt and, of course, it didn't work. My students had a great time razzing me about that for the rest of the class!

As with Windows, you can perform file system manipulation tasks either from the shell prompt or from within the Linux GUI. Many who are coming to Linux from a Windows background are tempted to perform these tasks solely using the GUI utilities provided with most distributions.

While these graphical utilities are handy and easy to use, you still need to learn how to do these tasks from the shell prompt. I realize that this goes against standard practice with Windows systems. I've worked with many Windows users who don't know the command prompt even exists on their systems. With Linux, however, you need to first become proficient with shell commands and then use the GUI utilities for convenience. I say this for the following reasons: ¹

1 Most employers and coworkers won't take you seriously as a Linux administrator if you can't manage the system from the shell prompt. It just goes with the territory. Many Linux systems, especially those deployed as servers, don't run a GUI. Running a GUI environment requires a lot of CPU overhead. Many server admins prefer to allocate those CPU cycles to system services instead of moving the mouse cursor on the screen or running a screensaver. In this situation, you need to know how to do things from the shell prompt.

- You need to know how to complete these tasks from the shell prompt to pass your Linux exam.



EXAM TIP For your exam, there are a large number of file system manipulation commands that you must be proficient with. You need to be able to do the following from the shell prompt:

- Navigate the file system
- Manage files and directories
- Run executable files
- Search the file system

I strongly recommend that you practice working with these commands until they become second nature to you. You'll be glad you did when you take your exam!

Let's begin by learning how to find files in the file system using shell prompt commands.

Finding Files in the Linux File System

One of the great weaknesses in my life is the fact that I have a terrible memory. You can call me on the phone and relate some bit of important information and I am pretty much guaranteed to forget what you told me by the time I hang up. This problem can be very embarrassing when I run into someone I've known and worked with for years and find myself unable to recall that person's name.

This fallibility, unfortunately, carries over into my work life. I can't tell you how many times I've created a file, saved it, and then promptly forgot where I saved it. To help people like me, Linux includes utilities that you can use to search for files in the file system. In this part of the chapter, you'll learn the following:

- Using `find`
- Using `locate`
- Using `which`
- Using `whereis`
- Using `type`

Let's begin by learning how to use the `find` utility.

Using `find`

The `find` utility is a fantastic tool that you can use to search the Linux file system. To use `find`, simply enter `find path -name "filename"` at the shell prompt. Replace *path* with the place in the file system where you want to start looking. Replace *filename* with the name of the file you want to search for.



TIP Don't forget to use quotation marks!

You can use regular expressions such as `*` to broaden your search results. For example, suppose you wanted to find all of the log files stored in your file system that have a `.log` extension. You could enter `find / -name "*.log"`. The `find` utility would then locate all files with an extension of `.log` starting in the root directory of the file system and searching all subdirectories. This is shown in Figure 4-7.

The `find` utility is flexible. You can also use the `-user "username"` option to search for files owned by a specific user, or use the `-size "size"` option to search for files of a specified size. You can use a `+` sign before the size value to search for files larger than the specified size, or a `-` sign before the size value to search for files smaller than the specified size. The `find` utility has many other options.



NOTE You can use the man page for `find` to see everything it has to offer.

In addition to `find`, you can also use the `locate` utility to search for files in tire file system.

Using locate

The `locate` utility functions in much the same manner as `find`. However, it has one distinct advantage over `find`. Here's the deal: Whenever you run a search with `find`, it manually walks through each directory in the path you specify in the command looking for matching files. This process can take a really long time, especially if you're doing a top-down search from the root of the file system.

The `locate` utility, on the other hand, works in a much more efficient way. Instead of walking the file system each time you run a search, it builds an index of the files in the file system. Then, when you run a search, `locate` simply queries tire index. It doesn't actually search the file system directly. The result is that `locate` runs *much* faster than `find` in most situations.

Figure 4-7
Using the `find` utility
to search for files

```
fs2:/ # find / -name "*.log"
/home/rtracy/.local/share/gvfs-metadata/label-VMware\x20Tools-29c19696.log
/home/rtracy/.local/share/gvfs-metadata/root-506bac77.log
/home/rtracy/.local/share/gvfs-metadata/home-d619f68d.log
/home/rtracy/.local/share/tracker/tracker-miner-fs.log
/home/rtracy/.local/share/tracker/tracker-extract.log
/home/rtracy/.local/share/tracker/tracker-store.log
/home/rtracy/.config/tomboy/tomboy-panel.log
/home/rtracy/.config/tomboy/tomboy.log
/root/.local/share/tracker/tracker-extract.log
/root/.local/share/tracker/tracker-miner-fs.log
/root/.local/share/gvfs-metadata/root-34fbe286.log
/root/.local/share/gvfs-metadata/home-7adc33eb.log
/root/.config/tomboy/tomboy-panel.log
/root/.config/tomboy/tomboy.log
```

To use locate, you must first install the findutils-locate package on your system. This process is shown on an openSUSE Linux system using the YaST utility in Figure 4-8.



NOTE We'll discuss how to install packages in more detail in a later chapter in this book.

With the package installed, an index (named locatedb) of all the files in your file system will be created in /var/log. This index will be updated each day with the latest changes to the file system. However, you can also manually update the index using the updatedb command from the shell prompt. Be aware that updatedb does all the legwork for the locate utility; it can take some time to complete and will use up a lot of system resources in the process.

With the index updated, you can search for files by simply entering `locate filename` at the shell prompt. For example, if you wanted to search for a file named `snmp.conf`, you could enter `locate snmp.conf`. This is shown here:

```
fs2:/ # locate snmp.conf
/etc/cups/snmp.conf
/usr/share/man/man5/cups-snmp.conf.5.gz fs2:/ #
```

Notice that locate found two files that had the text string `snmpd.conf` somewhere in the filename. With this in mind, let's next discuss how to use which.

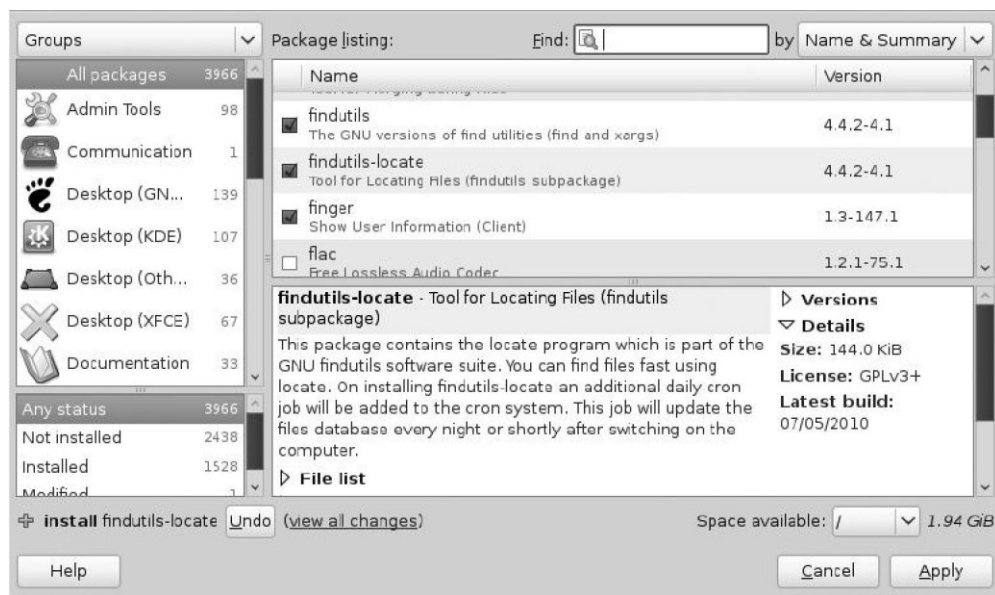


Figure 4-8 Using YaST to install the findutils-locate package

Using which

The which command is used to display full path to a shell command or utility. For example, if you wanted to know the full path to the ls command, you would enter which ls at the shell prompt. When you do, the full path to ls is displayed on screen, as shown here:

```
fs2:/ # which ls
/bin/ls
fs2:/ #
```

In addition to which, you can also use whereis. Let's talk about how to use this utility next.

Using whereis

The whereis command locates the source code, binary files, and manuals pages for specified files. The -b option returns the location of the binaries for the specified command. The -m option returns the location of the command's manual pages. To see the location of the command's source code, use the -s option with the whereis command. If no option is used, all information is returned.

For example, if you enter whereis -b ls at the shell prompt, the location of the ls command's binary executable file is returned. An example is shown here:

```
fs2:/ # whereis -b ls
ls: /bin/ls
```

Likewise, the whereis -m ls command returns the location of the ls command's man page file; as shown next: fs2:/ # whereis -m ls

```
ls: /usr/share/man/man1p/ls.1p.gz /usr/share/man/man1/ls.1.gz
```

Using type

The type command returns what type of command is executed when you enter it. The values returned will be one of the following:

- **A command that is hard coded into the shell itself** For example, the cd command is a built-in shell command:

```
fs2:/ # type cd
cd is a shell builtin
```
- **An external command that is called by the shell** If the command has been run recently in the shell, the output of type says that the command is *hashed*. This indicates the location of the external command in the file system has been stored in the shell's hash table. If you run the same command again, the shell will execute it without searching for it in the directories in the PATH environment variable. An example is shown here:

```
fs2:/ # type cat
cat is hashed (/bin/cat)
```

- **An alias** An alias is essentially a shortcut to another file in the Linux file system. Aliases are frequently used for commonly used commands, such as the `ls` command. This is shown in this example:

```
fs2:/ # type ls
ls is aliased to 'ls $LS_OPTIONS'
```



NOTE I discussed aliases in Chapter 2.

- **A function** The bash shell allows you to create new internal commands by defining your own shell functions.

You can also use the `-a` option with the `type` command. This option causes `type` to return all instances of the specified command in the file system. For example, entering the `type -a ip` command shows that there are two executables named `ip`:

```
fs2:/ # type -a ip
ip is /sbin/ip
ip is /bin/ip
```

Let's practice using Linux search tools in the following exercise.

Exercise 4-1: Using Linux Search Tools



VIDEO For more information watch *Exercise 4-1*.

In this exercise, you will practice using Linux search tools. Complete the following:

1. Verify that you are logged in to your Linux system.
2. If necessary, open a terminal session.
3. Change to your root user account by entering `su -` at the shell prompt and entering your root user's password.
4. Search for a file named `inittab` by entering `find / -name "resolve.conf"` at the shell prompt. Where is this file located?
5. Perform the same search using `locate` by entering `locate resolve.conf` at the shell prompt.



NOTE The `locate` command will not work if the `findutils-locate` package hasn't been installed on your system. Don't worry if this is the case; you'll learn how to install it later in this book.

6. Find the location of the `init` executable by entering `which init` at the shell prompt. Where is it located?
7. Enter `whereis init` at the shell prompt. Where is the file that contains the manual sections for the `init` command located?

8. At the shell prompt, display the last few lines of your messages log file by entering `tail /var/log/messages`.
9. At the shell prompt, see if the `tail` command is now hashed by the shell session by entering `type tail`. Is it? What does this mean?

Managing Directories from the Command Line

Now that you understand how the Linux file system is structured as well as how to search for files, you're ready to start managing the file system. In this part of the chapter, we're going to focus on managing directories from the shell prompt. The following topics are addressed:

- Navigating the file system
- Viewing directory contents
- Creating new directories
- Copying, moving, and deleting directories

Let's begin by discussing how to navigate the file system from the shell prompt.

Navigating the File System

As you work with the Linux file system from the shell prompt, one of the most common tasks you will complete is to move around between the different directories on your storage devices. Your Linux system provides the following shell commands that you can use to do this:

- `pwd`
- `cd`

Let's start by learning how to use the `pwd` command.

Using the `pwd` Command

The `pwd` command is a relatively simple utility. It simply displays the current directory on the screen.



NOTE The name `pwd` stands for “present working directory.”

This utility can be useful if your shell profile hasn't been configured to display the current directory as a part of the shell prompt. To use `pwd`, simply enter `pwd` at the shell prompt; as shown here:

```
rtracy@fs2:~> pwd
/home/rtracy
```

In this example, you can see that the current directory is set to `/home/rtracy`.



TIP The ~ character in the shell prompt points to the current user's home directory. Because I'm logged in as the rtracy user in the preceding example, ~ points to the /home/rtracy directory.

If you use the -P option with the pwd command, pwd will display the physical directory without symbolic links. We'll discuss link file later in this chapter.

At this point, you know how to identify what directory you're in. More than likely, you're going to need to change to a different directory in the file system at some point. Let's discuss how to do this next.

Using the cd Command

The cd command is used from the Linux shell prompt to change directories in the file system. To use this command, simply enter cd followed by the name of the directory you want to switch to. There are two ways you can do this. If you enter **cd** and then the name of a directory *without* specifying the full path to the directory, cd will assume that the directory is a subdirectory of the current directory. In the example that follows the **cd Documents** command has been issued:

```
rtracy@fs2:~> cd Documents/
rtracy@fs2:~/Documents>
```

Because the command was issued from /home/rtracy, cd changed directories to /home/rtracy/Documents. This is called using a *relative path*. The path specified with the command is relative to some other point in the file system, in this case, the /home/rtracy directory.

You can also use *absolute paths* with the cd command. When you use an absolute path, you specify the full path, starting from /, to the directory you want to change to as in the following example:

```
rtracy@fs2:~> cd /var/log
rtracy@fs2:/var/log>
```

Because I specified an absolute path (/var/log), the cd command knew that the /var/log directory didn't exist in the current directory. Instead, the path was determined from the root directory (/) and the current directory was changed to it.



TIP If you enter **cd** at the shell prompt without specifying a path, it will automatically change to the home directory of the currently logged-in user.

You can also use the cd command to move up the file system hierarchy. You can do this by entering **cd ..**. This will change the current directory to the next directory higher in the hierarchy. Consider the following example:

```
rtracy@fs2:/var/log> cd ..
rtracy@fs2:/var>
```

In this example, the `cd ..` command has been issued, changing from `/var/log` to the `/var` directory. To go up two directories, you can enter the `cd ../../` command at the shell prompt:

```
rtracy@fs2:/var/log> cd ../../
rtracy@fs2:/>
```



TIP You can also use the `..` alias to move up one level in the hierarchy and the `...` alias to move up two levels. Most distributions define these aliases automatically for you in one of your bash configuration files.

Let's next look at the `ls` command.

Viewing Directory Contents

To this point, you've learned how to view the current directory and change to other directories in the file system. Now you need to learn how to list the files and subdirectories that may exist within a directory. This is done using the `ls` command. If you enter `ls` at the shell prompt, the contents of the current directory are listed on the screen, as shown here:

```
rtracy@fs2:~> ls
bin      Documents Music      Public      Templates  test.txt  words
Desktop  Downloads Pictures  public_html test2.txt  Videos   yourfile.txt
rtracy@fs2:~>
```

As with the `cd` command, you can also provide an absolute path when using `ls`. This will cause `ls` to display the contents of the directory you specify. Consider this example:

```
rtracy@fs2:~> ls /var/log
acpid      cups      mail      ntp      Xorg.0.log.old
apparmor   faillog   mail.err   pk_backend_zypp  YaST2
audit      firewall  mail.info  pm-powersave.log zypp
boot.log    gdm       mail.warn  samba     zypper.log
boot.msg    krb5      messages   warn
boot.omsg   lastlog   NetworkManager  wtmp
ConsoleKit localmessages news      Xorg.0.log
rtracy@fs2:~>
```

In this example, the `ls /var/log` command has been issued, causing the contents of `/var/log` to be displayed. When working with `ls`, you can use a variety of options to customize how it works. Some of these options include the following:

- **-a** Displays all files, including hidden files. In the next example, the `ls -a` command has been issued in the `/home/rtracy` directory:

```
rtracy@fs2:~> ls -a
.      .esd_auth      .inputrc      test2.txt
..     .fontconfig     .local        test.txt
.bash_history .fonts         .mozilla      .themes
```

```
.bashrc bin      .gconf          Music           .thumbnails
.cache          .gconfd         .nautilus       Videos
.config .       .gnome2         Pictures        .viminfo
dbus Desktop   .gnome2_private .profile        .vimrc
. dmr          .gstreamer-0.10 Public          words
Documents      .gtk-bookmarks  public_html     .xim.template
Downloads      .gvfs .hplip      .pulse         .xinitrc.template
.emacs         .ICEauthority   .pulse-cookie   .xsession-errors
rtracy@fs2:~: .icons      .recently-used.xbel .xsession-errors.old
                                     Templates        yourfile.txt
```

- **-l** Displays a long listing of the directory contents. This is a very useful option. You can use it to see the filenames, ownership, permissions, modification dates, and sizes. A sample is shown here:

```
rtracy@fs2:~> ls -l
```

```
total 56
drwxr-xr-x 2 rtracy users 4096 2011-01-19 10:41 bin
drwxr-xr-x 2 rtracy users 4096 2011-01-19 10:42 Desktop
drwxr-xr-x 2 rtracy users 4096 2011-01-19 10:42 Documents
drwxr-xr-x 2 rtracy users 4096 2011-01-19 10:42 Downloads
drwxr-xr-x 2 rtracy users 4096 2011-01-19 10:42 Music
drwxr-xr-x 2 rtracy users 4096 2011-01-19 10:42 Pictures
drwxr-xr-x 2 rtracy users 4096 2011-01-19 10:42 Public
drwxr-xr-x 2 rtracy users 4096 2011-01-19 10:41 public.html
drwxr-xr-x 2 rtracy users 4096 2011-01-19 10:42 Templates
-rw-r--r- 1 rtracy users 37 2011-01-20 11:04 test2.txt
-rw-r--r- 1 rtracy users 182 2011-01-21 11:48 test.txt
drwxr-xr-x 2 rtracy users 4096 2011-01-19 10:42 Videos
-rw-r--r- 1 rtracy users 23 2011-01-20 11:32 words
-rw-r--r- 1 rtracy users 121 2011-01-21 11:46 yourfile.txt
```

```
rtracy@fs2:~>
```

- **-R** Displays directory contents recursively; that is, it displays the contents of the current directory as well as the contents of all subdirectories. Depending on the number of entries in the directory, you may want to append **|** more after using this option. This will cause the more utility to pause the display one page at a time. An example is shown in Figure 4-9, where the **ls -R** command has been issued from the **/var/log** directory.

This list is only a sampling of the different options you can use with **ls**. You can view the **ls** man page or info node to learn more.



TIP Most distributions define several aliases that you can use for the **ls** command. For example, on openSUSE the **dir** or **ll** alias runs the **ls -l** command while the **la** alias runs the **ls -la** command.

Let's practice navigating the file system in the following exercise.

```

messages
NetworkManager
news
ntp
pk_backend_zypp
pm-powersave.log
samba
warn
wtmp
Xorg.0.log
Xorg.0.log.old
YaST2
zypp
zypper.log

./ConsoleKit:
history

./cups:
access_log

./zypp:
history
rtracy@fs2: /var/log>

```

Figure 4-9 Using the `-R` option with the `ls` command

Exercise 4-2: Navigating the File System



VIDEO For more information watch *Exercise 4-2*.

In this exercise, you will practice using shell commands to navigate the Linux file system. Complete the following:

1. Boot your Linux system and log in as a standard user.
2. If necessary, open a terminal session.
3. Determine your current working directory by entering `pwd` at the shell prompt. What's the current directory?
4. Change directories to `/etc` by entering `cd /etc` at the shell prompt.
5. Generate a listing of the current directory by entering `ls`.
6. Generate a long listing of the current directory by entering `ls -l`.
7. Generate a long list and pause the output a page at a time by entering `ls -l | more`. Page your way through the listing.
8. Switch back to your home directory by entering `cd ~`.
9. Enter `ls`.
10. View the hidden files in your user's home directory by entering `ls -a`. Which files are displayed by `ls -a` that are not displayed by `ls`?

Let's now shift gears and discuss shell commands that you can use to manage directories in the Linux file system. Let's first look at how you create new directories.

Creating New Directories

You can use shell commands to create new directories. This is done using the **mkdir** command. You enter **mkdir** from the shell prompt followed by the name of the directory you want to create. In the next example, the **mkdir MyFiles** command has been issued from the home directory of the **rtracy** user.

```
rtracy@fs2:~> mkdir MyFiles
rtracy@fs2:~> ls
bin      Downloads  Pictures   Templates  Videos
Desktop  Music      Public     test2.txt   words
Documents MyFiles    publichtml test.txt    yourfile.txt
rtracy@fs2:~>
```

Notice in the output from the **ls** command that a new directory named **MyFiles** was created in **/home/rtracy**. Of course, you could use an absolute path with the directory name if you wanted to create it somewhere other than the current directory. For example, if you wanted to create a new directory named **backup** in the **/tmp** directory, you would enter **mkdir /tmp/backup** at the shell prompt.



TIP On many distributions, an alias named **md** is defined by one of the shell configuration files that runs the **mkdir -p** command. The **-p** option specifies that the entire directory path specified in the command be created if it doesn't exist. For example, **md ~/temp/backups/daily** would create the **temp** and **backups** directories, if they didn't already exist, before creating the **daily** directory.

Copying, Moving, and Deleting Directories

In addition to creating and viewing directories in the Linux file system, you can also copy, move, or delete them using shell commands. You use the following utilities to accomplish these tasks:

- **cp** This utility is used to copy entire directory structures from one location in the file system to another.

To copy an entire directory structure, you need to include the **-R** option, which specifies that the directory contents be recursively copied. In the example that follows the **cp -R ~/MyFiles ~/backup** command was issued in the **tux** user's home directory. This caused the **MyFiles** directory and all of its files and subdirectories to be copied to the **backup** directory in the user's home directory.



NOTE Because **cp** copies the directory, the original directory is left intact.

```
rtracy@fs2:~> cp -R ~/MyFiles ~/backup
rtracy@fs2:~> ls
backup  Documents  MyFiles    public_html  test.txt    yourfile.txt
```

```

bin      Downloads Pictures Templates  Videos
Desktop Music      Public   test2.txt  words
rtracy@fs2:~>

```

- **mv** The mv command is used much like cp. However, it copies the specified directory to the new location in the file system and then *deletes* the original. For example, to move a directory named backup from the home directory to /tmp, you would enter mv ~/backup /tmp. The mv command can also be used to rename directories. Simply enter mv followed by the directory to be renamed and then the new directory name. For example, to rename the backup directory in the home directory to temp, you would enter mv ~/backup ~/temp.
- **rmdir** This utility can be used to delete an existing directory. To use it, simply enter rmdir *directory_name*—for example, rmdir MyFiles. Be aware, however, that rmdir requires that the directory be empty before it will delete it.



TIP On many distributions, an alias named **rd** is defined by one of your shell configuration files that runs the **rmdir** command.

- **rm** The rm utility is a more powerful deletion utility that can be used to delete a populated directory. To delete a directory, enter rm -r *directory_name*.

So, that's how you manage directories from the shell prompt. With this in mind, I next need to discuss how to manage files within those directories from the shell prompt. Let's do that next.

Managing Files from the Command Line

In addition to managing directories in the file system, there will be many occasions when you need to manage the files they contain. In this part of the chapter, you will learn how to complete the following tasks:

- Creating files
- Viewing file contents
- Deleting files
- Copying and moving files
- Determining the file type

Let's first look at how to create new files in your file system.

Creating New Files

From time to time, you will need to create new files in your Linux file system. Creating one can be accomplished using the **touch** command from the shell prompt. To use touch, enter touch followed by the name of the file you want to create. In the next

example, the command `touch myfile.txt` was issued from within the home directory of the `rtracy` user:

```
rtracy@fs2:~> touch myfile.txt
rtracy@fs2:~> ls -l my*
-rw-r--r- 1 rtracy users 0 2011-02-01 11:36 myfile.txt
rtracy@fs2:~>
```

After entering `touch`, the `ls` command was entered. You can see that a 0-byte file was created named `myfile.txt` in `/home/rtracy`. If you wanted to create the file elsewhere in the file system, you would use an absolute path with the filename.

Viewing File Contents

As we discussed earlier, your Linux system and the services that run on it are configured using simple text files stored in (usually) the `/etc` directory in the file system. In addition, many of your system log files are saved as text files. Because Linux uses text files to configure and manage just about everything, you will frequently need to view the contents of files.

In the preceding chapter, you learned how to use text editors to open a file for viewing and editing. This works very well. However, there will be many occasions when you simply want to quickly view a text file on screen and don't need or want to load up a text editor. Linux provides you with a variety of command-line tools that you can use to do this. These include the following:

- **cat** The `cat filename` command will display the specified text file on screen. For example, if you needed to view the `/etc/xinetd.conf` configuration file, you would enter `cat /etc/xinetd.conf` at the shell prompt.



TIP The `cat` command doesn't pause the output, so if you use it to view a long file you may need to pipe it to the `more` program (`| more`) to pause the output a page at a time.

- **less** The `less filename` command can also be used to display the specified text file on screen, much like `cat`. However, the `less` command automatically pauses a long text file one page at a time. You can use the `SPACEBAR`, `PAGE UP`, `PAGE DOWN`, and `ARROW` keys to navigate around the output.
- **head** The `head filename` command is used to display the first couple of lines of a text file on the screen.
- **tail** The `tail filename` command is used to display the last couple of lines of a text file on screen. The `tail` command is particularly useful when displaying a log file on screen. When viewing a log file, you usually only need to see the end of the file. You probably don't care about log entries made several days or weeks ago. You can use `tail` to see just the last few log entries added to the end of the file.

The **tail** command also includes the **-f** option, which is very useful. You can use it to monitor the file specified in the command. If new content is added to the end of the file (such as a log file) the new lines will be displayed on screen. In Figure 4-10, the **tail -f /var/log/messages** command has been issued to monitor the file for new entries.

Using **-f** with the **tail** command can be very helpful when troubleshooting a misbehaving service or configuration problem on the system. When you're done, you can break out of **tail** by pressing **CTRL-C**.

In addition to creating and viewing files, you will also need to know how to delete existing files. Let's discuss how this is done next.

Deleting Files

As with directories, there will be times when you will need to delete an existing file from the Linux file system. Deleting files can be accomplished the **rm** command. This is the same command we used previously to delete directories. The **rm** utility is a powerful deletion utility that can be used to delete either a file or a populated directory. To delete a file, simply enter **rm filename**. In the next example, the **myfile.txt** file is deleted using the **rm** command:

```
rtracy@fs2:~> rm myfile.txt
rtracy@fs2:~>
```

You need to be very careful when using the **rm** utility! As you can see in the preceding example, it doesn't prompt you to confirm a deletion operation. It assumes that you really meant to delete the file or directory and does the deed. And by the way, the deleted file doesn't get moved to the Recycle Bin. If you delete a file or directory with **rm**, it is gone! If you want **rm** to prompt you before deleting a file or directory, include the **-i** option.

Let's next review how you can copy and move files from the shell prompt.

```
fs2:~ # tail -f /var/log/messages
Feb  1 10:30:34 fs2 kernel: [ 7065.478338] VMCIUtil: Updating context id from 0x3a5e93d3 to 0x3a5e93d3 on event 0.
Feb  1 10:35:27 fs2 kernel: [ 7358.121098] VMCIUtil: Updating context id from 0x3a5e93d3 to 0x3a5e93d3 on event 0.
Feb  1 11:06:54 fs2 unix2_chkpwd: gkr-pam: unlocked login keyring
Feb  1 11:17:12 fs2 rsyslogd: -- MARK --
Feb  1 11:21:36 fs2 unix2_chkpwd: gkr-pam: unlocked login keyring
Feb  1 11:36:03 fs2 unix2_chkpwd: gkr-pam: unlocked login keyring
Feb  1 11:47:09 fs2 unix2_chkpwd: gkr-pam: unlocked login keyring
Feb  1 11:47:20 fs2 su: The gnome keyring socket is not owned with the same credentials as the user login: /tmp/keyring-TMU5RA/control
Feb  1 11:47:20 fs2 su: gkr-pam: couldn't unlock the login keyring.
Feb  1 11:47:20 fs2 su: (to root) rtracy on /dev/pts/0
```

Figure 4-10 Using **tail** with the **-f** option to monitor a log file

Copying and Moving Files

In addition to creating and deleting files in the Linux file system, you can also copy or move them. You use the same utilities that you used to manage directories to copy or move files:

- **cp** This utility is used to copy files from one location in the file system to another. For example, to copy a file named `/tmp/schedule.txt` to your home directory, you would enter **cp /tmp/schedule.txt ~**. Remember, **cp** makes a copy of the file, so the original file is left intact. You can use the **-R** option with **cp** to copy files recursively.
- **mv** The **mv** command copies a file to the new location in the file system and then deletes the original. For example, to move a file named `mylog.txt` from `/tmp` to `/var/log`, you would enter **mv /tmp/mylog.txt /var/log**. As with directories, the **mv** command can also be used to rename files. Simply enter **mv** followed by the file to be renamed and then the new filename. For example, to rename `schedule.txt` to `schedule.old`, you would enter **mv schedule.txt schedule.old**.

Determining the File Type

You can use the **file** command at the shell prompt to determine a file's type. Understand that, on Linux, file extensions, such as `txt`, are not necessary. Therefore, it may not be immediately obvious from a file's name what kind of file you are looking at.

The **file** command can be used at the shell prompt to get information about a file. The output of the **file** command will not only tell you the general file type (such as directory, named pipe, link, etc.), but it can also provide you with detailed information about your files. To do this, it compares the file you specify in the command with file signatures contained in `/usr/share/misc/magic`, `/usr/share/misc/magic.mgc`, and `/etc/magic`.

In the next example, you can see that the **file** command has identified the `words` file in my home directory as an ASCII text file while the `resources` file is an OpenOffice.org document:

```
rtracy@fs2:~> ls
bin      Downloads      MyFiles      public_html  Templates  Videos
Desktop  hs_err_pid7458.log Pictures      resources    test2.txt  words
Documents Music          Public      temp         test.txt   yourfile.txt
rtracy@fs2:~> file words
words: ASCII text
rtracy@fs2:~> file resources
resources: OpenDocument Text
rtracy@fs2:~>
```

Thus far, we've been working primarily with files, directories, and aliases in the Linux file system. However, there's a fourth type of Linux file that is used a lot and that you need to be familiar with. These files are called *links*. Let's discuss how they work next.

Working with Link Files

As we discussed earlier in this chapter, the Linux file system supports a file type called a *link file*. Link files don't contain content in the way that regular files do. Instead, they are redirectors that point you to a different file or directory in the file system. On Linux you can create two different types of link files:

- **Hard** A hard link is a file that points directly to the inode of another file. An *inode* stores basic information about a file in the Linux file system, including its size, device, owner, and permissions. Because the two files use the same inode, you can't actually tell which file is the pointer and which is the pointee after the hard link is created. It is as if the two files were the same file, even though they may exist in different locations in the file system.
- **Symbolic** A symbolic link file also points to another file in the file system. However, a symbolic link has its own inode. Because the pointer file has its own inode, the pointer and the pointee in the file system can be easily identified. For example, in the preceding chapter, you saw that the `/usr/bin/vi` file is actually a symbolic link that points to the `/bin/vim` file. This is shown in this example:

```
fs2:/usr/bin # ls -l vi
lrwxrwxrwx 1 root root 8 Jan 19 10:24 vi -> /bin/vim
fs2:/usr/bin #
```

To create a link file, you use the `ln` command. The syntax is **`ln pointee_file pointer_file`**. Using `ln` without any options creates a hard link. If you want to create a symbolic link, you use the `-s` option. In the example that follows, the `ln` command has been used to create a symbolic link between a file named `myapp` in the `bin` subdirectory of my user's home directory and an executable file named `myapp` located in `/var/opt`:

```
rtracy@fs2:~/bin> ln -s /var/opt/myapp
myapp rtracy@fs2:~/bin> ls -l total 0
lrwxrwxrwx 1 rtracy users 14 2011-02-01 13:15 myapp -> var/opt/myapp
rtracy@fs2:~/bin> cd ..
rtracy@fs2:~> myapp
This is my new executable script file,
rtracy@ fs2:~>
```

Using the `ls -l` command, you can see that `myapp` actually points to the `/var/opt/myapp` executable file. This can be useful trick if the `/var/opt/myapp` program is one that I need to use frequently. Because the `/var/opt` directory is not in my user's `PATH` environment variable, I would need to supply the full path to the `myapp` executable each time I wanted to run it (or switch to the `/var/opt` directory first). However, the `bin` subdirectory of my home directory is automatically added to the `PATH` environment variable by my bash configuration files. By placing the symbolic link in this directory, I can run the `/var/opt/myapp` executable by simply entering `myapp` at the shell prompt. The symbolic link file redirects the shell to the appropriate file in `/var/opt` and runs it from there. This is shown in the preceding example.



NOTE Later in this book, we'll discuss the vital role symbolic links play when the init process initializes your Linux system.

Let's practice managing files in the following exercise.

Exercise 4-3: Managing Files and Directories



VIDEO For more information watch *Exercise 4-3*.

In this exercise, you will practice managing files and directories using shell commands. Complete the following:

1. With your Linux system running and authenticated as a regular user, open a terminal session and switch to your home directory by entering `cd` at the shell prompt.
2. Create a new directory in your home directory by entering **`mkdir MyFiles`**.
3. Enter **`ls`** and verify that the new directory exists.
4. Create a new file in the MyFiles directory called `myfile.txt` by entering **`touch ~/MyFiles/myfile.txt`**.
5. Enter **`ls ~/MyFiles`** and verify that the file exists.
6. Change to your root user account by entering **`su -`** and entering your root user's password.
7. View your system log file by entering **`cat /var/log/messages | more`**.
8. Page through a few pages of the file, and then press CTRL-C to break out to the shell prompt.
9. View the last few entries in the log file by entering **`tail /var/log/messages`**.
10. Monitor your `/var/log/messages` log file while you stop and start services by doing the following:
 - a. At the shell prompt, enter **`tail -f /var/log/messages`**. You should see that the log file is now being monitored by tail.
 - b. Open a new terminal window and switch to your root user account using the `su -` command and your root user's password.
 - c. Disable your system's network card by entering **`ifdown eth0`** at the shell prompt.
 - d. Switch to the terminal session where tail is running. You should see new lines added to the file as the network board is disabled.
 - e. Switch back to the second shell prompt and enter **`ifup eth0`**. Again, you should see new lines added to the file as the network board is re-enabled in the first terminal window.

- f. Enter `exit` twice to close the second terminal window.
- g. Switch to the terminal session where `tail` is running and press `CTRL-C` to break out of `tail`.
11. Switch back to your regular user account by entering `exit`.
12. Delete the `MyFiles` directory in your home directory by entering **`rm -r ~/MyFiles`**.
13. Enter **`ls`** and verify that the directory and its contents are gone.
14. Make a copy of your home directory files and subdirectories in `/tmp` by entering **`cp -R ~/tmp`**.
15. View the contents of `/tmp` by entering **`ls /tmp`**. Verify that your home directory was copied.
16. Work with symbolic links by doing the following:
 - a. Create a symbolic link from a directory named `docs` in your home directory to the `/usr/share/doc` directory by entering **`ln -s /usr/share/doc/ ~/docs`**.
 - b. Enter **`ls -l`**. Verify that the `docs` file points to `/usr/share/doc/`.
 - c. Enter **`cd docs`**.
 - d. Enter **`pwd`**. What directory are you in?
 - e. Enter **`ls`**. You should see the contents of the `/usr/share/doc/` directory even though you are still in `~/docs`.

Finding Content within Files

Earlier in this chapter, we discussed shell commands that can be used to search for files in the file system. However, Linux also provides a utility called **grep** that you can use to search for specific content *within* a file. The `grep` utility is extremely useful; in fact, I dare say that most Linux admins use it on a daily basis. Using `grep`, you can search through a file for a particular text string. This can come in handy when you want to search a very large log file for a specific message or when you need to find a specific directive within a configuration file. You can even use `grep` to search through multiple files at once to locate a particular expression.

To use `grep`, you enter **`grep search_expression file_name`** at the shell prompt. For example, let's suppose you want to search through your `/var/log/messages` file for any log entries related to the VNC service running on your Linux system. You would enter `grep vnc /var/log/messages`. The `grep` utility would then display each line from the file that contained your search term, as shown here:

```
fs2:~ # grep vnc /var/log/messages
Feb 1 13:47:25 fs2 xinetd[7134]: Reading included configuration file:
/etc/xinetd.d/vnc [file=/etc/xinetd.d/vnc] [line=15]
fs2:~ #
```

Likewise, if you needed to find a line in your `/etc/ntp.conf` file that contains the directive "server", you could enter **grep server /etc/ntp.conf** at the shell prompt. This is shown in the next example:

```
fs2:~ # grep server /etc/ntp.conf
# server 127.127.8.0 mode 5 prefer server 127.127.1.0
## # rcntp addserver <yourserver> server bigben.cac.washington.edu iburst
# key (7) for accessing server variables
# controlkey 15          # key (6) for accessing server variables
fs2:~ #
```

When working with `grep` at the command line, you can use the following options:

- **-i** Ignores case when searching for the search text.
- **-l** Displays only the names of the files that contain the matching text. It doesn't display the actual matching line of text.
- **-n** Displays matching line numbers.
- **-r** Searches recursively through subdirectories of the path specified.
- **-v** Displays all lines that *do not* contain the search string.

The `grep` utility works great when you need to search for a specific text string within a file. However, there may be times when you need to search for more complex patterns. In this situation, you can use the **egrep** command instead.



NOTE The `egrep` command is the same as using the **grep -E** command.

The key advantage of `egrep` is that it can search files for extended *regular expressions*. Regular expressions are strings consisting of *metacharacters*, and *literals*.



NOTE Literals are regular characters (A-Z, a-z) and numerals.

Metacharacters are characters that do not represent themselves but instead represent other characters. They can also be used to specify a character's location within a text string. Some commonly used regular expressions are listed in Table 4-6. See the `regex(7)` man page for more information.

In addition to `egrep`, you can also use the **fgrep** (fixed-string `grep`) utility to search for content within files.



NOTE Running `fgrep` is the same as running **grep -F**.

Metacharacter	Function	Example
*	Matches any number of any characters.	Myfile* would match Myfile1, Myfile2, Myfiles, Myfiles23, etc.
.	Matches a single character	Myfile. would match Myfile1, Myfile2, and Myfiles, but would not match Myfiles23.
^	Matches an expression if it appears at the beginning of a line.	^server would match any instance of "server" as long as it appears at the beginning of a line.
\$	Matches an expression if it appears at the end of a line.	server\$ would match any instance of "server" as long as it appears at the end of a line.
	Matches the expressions on either side of the pipe character.	Server server would match either "Server" or "server".
[nnn]	Matches any one character between the braces.	[xyz] would match any one of "x", "y", or "z".
[^nnn]	Matches an expression that does <i>not</i> contain any one of the characters specified.	[^aei] would <i>not</i> match the characters "a", "e", or "i".
[n-n]	Matches any single character in the range.	[1-5] would match any number between 1 and 5.

Table 4-6 Regular Expressions

The **fgrep** utility searches files for lines that match a fixed string. Unlike **egrep**, it does not perform searches for regular expressions. Instead, **fgrep** uses direct string comparisons to find matching lines of text in the input. The syntax is **fgrep pattern filename**. For example, running the **fgrep server *.c** command searches for the string "server" in all files in the current directory whose filename has a ".c" extension.

Let's practice using Linux search tools in the following exercise.

Exercise 4-4: Using grep

In this exercise, you will practice using Linux search tools. Complete the following:

1. With your Linux system running and authenticated as a regular user, switch to root by entering **su -** followed by your root user's password at the shell prompt.
2. Locate references to the **inittab** file in your system log file by entering **grep inittab /var/log/messages**. What can you learn about this file from the output of **grep**?
3. Use **egrep** to find information about the **init** process in your system log file by entering **egrep init* /var/log/messages** at the shell prompt.
4. Use **egrep** again to find information about all the Ethernet interfaces installed in your system by entering **egrep eth[0-3] /var/log/messages** at the shell prompt.
5. Find all lines in your system log file that end with "Enabled" by entering **egrep Enabled\$ /var/log/messages** at the shell prompt.

You are now a file system pro! You have to be humble when you know this much!

TIP The `/etc/updatedb.conf` file is used to configure the behavior of the `updatedb` utility. You can set the `PRUNEFS` variable in this file to the file system type(s) you don't want `updatedb` to scan. You can also set the `PRUNEPATHS` variable in this file to a list of directories in your file system you want `updatedb` to skip when scanning.

Chapter Review

In this chapter, I reviewed a variety of topics related to managing the Linux file system. I began this chapter by introducing you to the Linux file system. I first differentiated between a general file system and a specific disk file system such as Reiser or ext4. I then pointed out that the role of the Linux file system is to organize data such that it can be easily located and retrieved as well as reliably preserved.

I then related that Linux uses a hierarchical file system. The topmost directory of this hierarchy is the root directory (`/`). We discussed the role of the various standard directories used in a typical Linux system as specified in the Filesystem Hierarchy Standard (FHS). These include:

- `/bin`
- `/boot`
- `/dev`
- `/etc`
- `/home`
- `/lib`
- `/media`
- `/mnt`
- **`/opt`**
- `/proc`
- `/root`
- `/sbin`
- `/srv`
- `/sys`
- `/tmp`
- `/usr`
- `/var`

I also discussed the four different types of files used on a Linux system: regular files, links, FIFOs, and sockets.

With this background information in hand, I turned our attention to common Linux file system tasks. I emphasized that, while most of these tasks can be performed using graphical utilities, the Linux exam expects you to know how to complete them from the shell prompt.

I first discussed utilities that you can use to search for files or directories in the Linux file system. I reviewed how to use the `find` and `locate` commands to find specific files or directories. I pointed out that `find` manually walks the file system hierarchy to conduct its searches, which can be somewhat slow. As an alternative, the `locate` utility creates a database of files in the file system. When you search with `locate`, it queries its database instead of walking the file system hierarchy, which is much faster.

I also pointed out that you can use the `which` command to find out where the executables for system utilities, such as `ls`, `cp`, and `man`, reside in the file system. I also looked at the `whereis` command, which displays information such as executable file, source file, and man page file locations for a particular shell command. I also reviewed how to use the `type` command to determine a file's type.

I next reviewed how to navigate through the Linux file system using the `pwd` and `cd` commands. I then discussed how to manage files and directories. I pointed out that you could create new files using the `touch` command. You can also create new directories using the `mkdir` command.

I also reviewed the utilities you can use to delete files and directories, including `rmdir` and `rm`. To copy and move files and directories, you can use the `cp` and `mv` utilities. I pointed out that `mv` can also be used to rename files or directories. I also discussed link files. I pointed out that link files point to other files or directories in the file system. I related that you can create either hard links or symbolic links using the `ln` command.

I then discussed utilities you can use to view the contents of text files on screen. I pointed out that you can use `cat`, `less`, `head`, or `tail` to do this. I also pointed out that you can use the `-f` option with `tail` to monitor changes to a text file, such as a log file, as they are made. I ended the chapter by discussing how to use `grep` to find content within text files.

Accelerated Review

- The role of the file system is to store and organize data such that it can be easily located and retrieved.
- The file system must also keep data intact.
- Linux uses a hierarchical file system.
- The Linux file system hierarchy is based on the Filesystem Hierarchy Standard (FHS).
- The topmost directory is `/`.
- Other standard directories are created beneath `/` and serve functions defined in the FHS.
- The Linux file system uses regular files, links, FIFOs, and sockets.
- Most file system management tasks can be completed using either graphical or command-line tools.

- You can use the find utility to locate files or directories in the file system.
- The find utility manually walks the file system hierarchy to search for files.
- You can use locate to search for files or directories.
- The locate utility maintains a database of all files in the file system.
- When locate conducts a search, it searches the database instead of the file system.
- The which command is used to display the location of files in the file system.
- The whereis command displays information about a command's executable file, source code file, and man page file.
- The type command displays a file's type.
- The pwd command is used to display the current directory.
- The cd command is used to change directories.
- The ls command is used to display directory contents.
- Using ls with the -l option displays additional details about files and directories.
- Using ls with the -R option displays directory contents recursively.
- The touch command is used to create new files.
- The mkdir command is used to create new directories.
- You can use cat to view a text file on screen.
- You can also use less to view a text file on screen.
- The less command pauses the display one line at a time.
- The head command can be used to display the first few lines of a text file.
- The tail command can be used to display the last few lines of a text file.
- The tail command used with the -f option can monitor a text file for changes.
- You can use rmdir to delete an empty directory.
- You can use rm to delete populated directories or files.
- The cp command is used to copy files.
- The mv command is used to move files.
- The Linux file system allows you to create link files that point to other files or directories in the file system.
- Hard links point directly to the inode of another file.
- Symbolic links have their own inode.
- Links are created using the ln command.
- You can use grep to search for text within a file